torch.jit.freeze#

https://pytorch.org/docs/stable/generated/torch.jit.freeze.html

Description:

Freeze ScriptModule, inline submodules, and attributes as constants. Freezing a ScriptModule will clone it and attempt to inline the cloned module's submodules, parameters, and attributes as constants in the TorchScript IR Graph. By default, forward will be preserved, as well as attributes & methods specified in preserved_attrs. Additionally, any attribute that is modified within a preserved method will be preserved. Freezing currently only accepts ScriptModules that are in eval mode. Freezing applies generic optimization that will speed up your model regardless of machine. To further optimize using server-specific settings, run optimize_for_inference after freezing. mod (ScriptModule) – a module to be frozen

Function Signature

```
torch.jit.freeze(mod, preserved_attrs=None,
optimize_numerics=True)[source]#

Parameters

Returns
```

Returns:

Frozen ScriptModule.

Code Examples

```
def forward(self, input):
        output = self.weight.mm(input)
        output = self.linear(output)
        return output

scripted_module = torch.jit.script(MyModule(2, 3).eval())
frozen_module = torch.jit.freeze(scripted_module)
# parameters have been removed and inlined into the Graph as
constants
assert len(list(frozen_module.named_parameters())) == 0
# See the compiled graph as Python code
print(frozen_module.code)
```

```
def forward(self, input):
        self.modified_tensor += 1
        return input + self.modified_tensor

scripted_module = torch.jit.script(MyModule2().eval())
frozen_module = torch.jit.freeze(scripted_module,
preserved_attrs=["version"])
# we've manually preserved `version`, so it still exists on the
frozen module and can be modified
assert frozen_module.version == 1
frozen_module.version = 2
# `modified_tensor` is detected as being mutated in the forward,
so freezing preserves
# it to retain model semantics
assert frozen_module(torch.tensor(1)) == torch.tensor(12)
# now that we've run it once, the next result will be
incremented by one
assert frozen_module(torch.tensor(1)) == torch.tensor(13)
```

Notes & Warnings

NOTE Note Freezing submodule attributes is also supported: frozen_module = torch.jit.freeze(scripted_module, preserved_attrs=[“submodule.version”])

NOTE Note If you’re not sure why an attribute is not being inlined as a constant, you can run dump_alias_db on frozen_module.forward.graph to see if freezing has detected the attribute is being modified.

NOTE Note Because freezing makes weights constants and removes module hierarchy, to and other nn.Module methods to manipulate device or dtype no longer work. As a workaround, You can remap devices by specifying map_location in torch.jit.load, however device-specific logic may have been baked into the model.

Detailed Documentation

Documentation

Freeze ScriptModule, inline submodules, and attributes as constants.

Freezing a ScriptModule will clone it and attempt to inline the cloned module’s submodules, parameters, and attributes as constants in the TorchScript IR Graph. By

default, forward will be preserved, as well as attributes & methods specified in preserved_attrs. Additionally, any attribute that is modified within a preserved method will be preserved.

Freezing currently only accepts ScriptModules that are in eval mode.

Freezing applies generic optimization that will speed up your model regardless of machine. To further optimize using server-specific settings, run optimize_for_inference after freezing.

mod (ScriptModule) – a module to be frozen

preserved_attrs (Optional[List[str]]) – a list of attributes to preserve in addition to the forward method. Attributes modified in preserved methods will also be preserved.

optimize_numerics (bool) – If True, a set of optimization passes will be run that does not strictly preserve numerics. Full details of optimization can be found at torch.jit.run_frozen_optimizations.

Example (Freezing a simple module with a Parameter):

Example (Freezing a module with preserved attributes)

Freezing submodule attributes is also supported: frozen_module = torch.jit.freeze(scripted_module, preserved_attrs=[“submodule.version”])

If you’re not sure why an attribute is not being inlined as a constant, you can run dump_alias_db on frozen_module.forward.graph to see if freezing has detected the attribute is being modified.

Because freezing makes weights constants and removes module hierarchy, to and other nn.Module methods to manipulate device or dtype no longer work. As a

workaround, You can remap devices by specifying map_location in torch.jit.load, however device-specific logic may have been baked into the model.
